

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Яндекс.Контеста №1
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

A. Агроном-любитель	3
Условие:	3
Решение:	4
О решении:	5
B. Зоопарк Глеба	6
Условие:	6
Решение:	7
О решении:	8
C. Конфигурационный файл	9
Условие:	9
Решение:	10
О решении:	11
D. Профессор Хаос	12
Условие:	12
Решение:	13
О решении:	14

А. Агроном-любитель

Условие:

Городской школьник Лёша поехал на лето в деревню и занялся выращиванием цветов. Он посадил n цветков вдоль одной длинной прямой грядки, и они успешно выросли. Лёша посадил множество различных видов цветков, i -й от начала грядки цветок имеет вид a_i , где a_i — целое число, номер соответствующего вида в «Каталоге юного агронома».

Теперь Лёша хочет сделать фотографию выращенных им цветов и выложить ее в раздел «мои грядки» в социальной сети для агрономов «ВКонтакте». На фотографии будет виден отрезок из одного или нескольких высаженных подряд цветков.

Однако он заметил, что фотография смотрится не очень интересно, если на ней много одинаковых цветков подряд. Лёша решил, что если на фотографии будут видны три цветка одного вида, высаженные подряд, то его друзья — специалисты по эстетике цветочных фотографий — поставят мало лайков.

Помогите ему выбрать для фотографирования как можно более длинный участок своей грядки, на котором нет трех цветков одного вида подряд.

Формат ввода

В первой строке содержится целое число n ($1 \leq n \leq 200\,000$) — количество цветов на грядке.

Во второй строке содержится n целых чисел a_i ($1 \leq a_i \leq 10^9$), обозначающих вид очередного цветка. Одинаковые цветки обозначаются одинаковыми числами, разные — разными.

Формат вывода

Выведите номер первого и последнего цветка на самом длинном искомом участке. Цветки нумеруются от 1 до n .

Если самых длинных участков несколько, выведите участок, который начинается раньше.

Пример

Ввод 

```
6
5 6 6 6 23 9
```

Вывод 

```
3 6
```

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int flowers_count, flower_type;
    cin >> flowers_count;

    vector<int> flower_types;

    for (int i = 0; i < flowers_count; i++) {
        cin >> flower_type;
        flower_types.push_back(flower_type);
    }

    int max_len = 2, curr_max_len = 2, start_index_max_len = 0, end_index_max_len = 1;

    if (flowers_count < 3) {
        cout << 1 << " " << flowers_count << endl;
        return 0;
    }

    for (int i = 0; i < flowers_count - 2; i++) {
        if (!(flower_types[i] == flower_types[i + 1] && flower_types[i + 1] ==
flower_types[i + 2])) {
            curr_max_len++;
        } else {
            if (curr_max_len > max_len) {
                max_len = curr_max_len;
                start_index_max_len = i - curr_max_len + 2;
                end_index_max_len = i + 1;
            }

            curr_max_len = 2;
        }
    }

    if (curr_max_len > max_len) {
        max_len = curr_max_len;
        start_index_max_len = flowers_count - curr_max_len;
        end_index_max_len = flowers_count - 1;
    }

    cout << start_index_max_len + 1 << " " << end_index_max_len + 1 << endl;
    return 0;
}
```

О решении:

Идея:

Идем по массиву и наращиваем длину подмассива до тех пор, пока не встретим 3 подряд идущих одинаковых элемента. Если встретили, то запускаем проверку, в которой обновляем длину наибольшего подмассива и индексов начала и конца этого подмассива только в том случае, если новый подмассив именно больше старого максимального (таким образом гарантируем, что при нескольких подмассивах максимально возможной длины в ответ выведутся только индексы начала и конца первого встретившегося). В худшем случае (и при условии, что длина хотя бы 3 элемента) максимальная длина подмассива будет 2, так как 2 подряд идущих одинаковых элемента являются корректным подмассивом.

Сложность по времени:

$O(n)$ – так как линейно проходимся по массиву.

Сложность по памяти:

$O(n)$ – так как храним массив длины n .

В. Зоопарк Глеба

Условие:

Недавно Глеб открыл зоопарк. Он решил построить его в форме круга и, естественно, обнёс забором. Глеб взял вас туда начальником охраны. Казалось бы все началось так хорошо, но именно в вашу первую смену все животные разбежались. В зоопарке n животных различных видов, также под каждый из видов есть свои ловушки. К сожалению некоторые животные враждуют с друг другом в природе (они обозначены разными буквами), а зоопарк обнесён забором и имеет форму круга. С помощью камер, удалось выяснить, где находятся все животные. Умная система поддержки жизнедеятельности зоопарка уже просканировала зоопарк и вывела id всех животных и ловушек в том порядке, в котором они видны из центра зоопарка. Получилось так, что все животные и все ловушки находятся на краю зоопарка. Вы хотите понять, могут ли животные прийти в свою ловушку так, чтобы их путь не пересекался с другими. Если да, также предъявите какую-нибудь из схем поимки животных.

Формат ввода

На вход подается строка из $2 \cdot n$ символов латинского алфавита, где маленькая буква - животное, а большая - ловушка. Размер строки не более 100000.

Формат вывода

Требуется вывести "Impossible", если решения не существует или "Possible", если можно вернуть всех животных в клетки. В случае если можно, то для каждой ловушки в порядке обхода требуется вывести индекс животного в ней.

Пример 1

Ввод 

ABba

Вывод 

Possible
2 1

Пример 2

Ввод 

ABab

Вывод 

Impossible

Решение:

```
#include <iostream>
#include <stack>
#include <vector>

using namespace std;

int main() {
    string line;
    cin >> line;

    int animals_count = line.size() / 2;
    stack<pair<int, char>> st;
    vector<int> animals_in_traps(animals_count + 1, 0);
    int animal_id = 0;
    int trap_id = 0;

    for (char ch : line) {
        if (islower(ch)) {
            animal_id++;
        } else {
            trap_id++;
        }

        if (!st.empty()) {
            char top_ch = st.top().second;
            bool same_letter = tolower(top_ch) == tolower(ch);
            bool lower_top_upper_curr = islower(top_ch) && isupper(ch);
            bool upper_top_lower_curr = isupper(top_ch) && islower(ch);
            bool diff_case = lower_top_upper_curr || upper_top_lower_curr;

            if (same_letter && diff_case) {
                if (islower(ch)) {
                    int trap_num = st.top().first;
                    animals_in_traps[trap_num] = animal_id;
                } else {
                    int animal_num = st.top().first;
                    animals_in_traps[trap_id] = animal_num;
                }

                st.pop();
                continue;
            }
        }

        if (islower(ch)) {
            st.push({animal_id, ch});
        } else {
            st.push({trap_id, ch});
        }
    }
}
```

```

if (st.empty()) {
    cout << "Possible\n";
    for (int i = 1; i < (int)animals_in_traps.size(); i++) {
        cout << animals_in_traps[i] << " ";
    }
} else {
    cout << "Impossible\n";
}
}

```

О решении:

Идея:

Требование непересечения траекторий животных на окружности можно трактовать, как задачу о непересекающихся хордах (так как каждому животному соответствует своя ловушка). Если мы представим окружность и хорды (траектории на окружности), то каждая хорда должна “стянуться”, не задев другие хорды. Для этой задачи будем использовать стек. В стек будем закидывать элементы без пары (пока что). Если текущий символ будет совпадать по значению буквы, а также он будет разных регистров с символом из стека, тогда эту пару мы считаем, как “стянувшуюся” хорду, элемент из строки никуда не добавляется, а элемент с вершины стека достаем. Если такого не произошло, то добавляем элемент на вершину стека. Если по окончании прохода стек пустой, то все пары корректно “стянулись”, животных поймать можно. В противном случае поймать их нельзя.

Для удобства определения порядка заведем два счетчика: для животных и ловушек, которые мы увеличиваем на один в зависимости от того, кого мы встретили. В стеке мы храним пары значений типа {номер объекта, символ}.
Случай 1: в стеке ловушка, встретили животное. Тогда в массив пойдет по индексу номера ловушки со стека идет это животное.

Случай 2: в стеке животное, встретили ловушку. Тогда в массив в текущую ловушку идет животное со стека.

Сопоставили – достаем элемент со стека и идем дальше.

В противном случае добавляем на вершину стека животное/ловушку.

Сложность по времени:

$O(n)$ – так как линейно проходимся по строке длины n .

Сложность по памяти:

$O(n)$ – так как в худшем случае в стеке хранятся все элементы строки + в массиве хранится порядок загона животных длины $n/2$.

С. Конфигурационный файл

Условие:

Вадим разрабатывает парсер конфигурационных файлов для своего проекта. Файл состоит из блоков, которые выделяются с помощью символов «{» — начало блока, и «}» — конец блока. Блоки могут вкладываться друг в друга. В один блок может быть вложено несколько других блоков.

В конфигурационном файле встречаются переменные. Каждая переменная имеет имя, которое состоит из не более чем десяти строчных букв латинского алфавита. Переменным можно присваивать числовые значения. Изначально все переменные имеют значение 0.

Присваивание нового значения записывается как `<variable>=<number>`, где `<variable>` — имя переменной, а `<number>` — целое число, по модулю не превосходящее 10^9 . Парсер читает конфигурационный файл построчно. Как только он встречает выражение присваивания, он присваивает новое значение переменной. Это значение сохраняется до конца текущего блока, а затем восстанавливается старое значение переменной. Если в блок вложены другие блоки, то внутри тех из них, которые идут после присваивания, значение переменной также будет новым.

Кроме того, в конфигурационном файле можно присваивать переменной значение другой переменной. Это действие записывается как `<variable1>=<variable2>`. Прочитав такую строку, парсер присваивает текущее значение переменной `variable2` переменной `variable1`. Как и в случае присваивания константного значения, новое значение сохраняется только до конца текущего блока. После окончания блока переменной возвращается значение, которое было перед началом блока.

Для отладки Вадим хочет напечатать присваиваемое значение для каждой строки вида `<variable1>=<variable2>`. Помогите ему отладить парсер.

Формат ввода

Входные данные содержат хотя бы одну и не более 10^5 строк. Каждая строка имеет один из четырех типов:

- { — начало блока;
- } — конец блока;
- `<variable>=<number>` — присваивание переменной значения, заданного числом;
- `<variable1>=<variable2>` — присваивание одной переменной значения другой переменной. Переменные `<variable1>` и `<variable2>` могут совпадать.

Гарантируется, что ввод является корректным и соответствует описанию из условия. Ввод не содержит пробелов.

Формат вывода

Для каждой строки типа `<variable1>=<variable2>` выведите значение, которое было присвоено.

Пример

Ввод	Вывод
a=b	0
b=123	123
var=b	-34
b=-34	1000000000
{	1000000000
c=b	123
b=1000000000	-34
d=b	
{	
a=b	
e=var	
}	
}	
b=b	

Решение:

```
#include <iostream>
#include <map>
#include <stack>
#include <string>
#include <vector>

using namespace std;

bool is_number(const string& s) {
    if (s[0] == '-') {
        for (int i = 1; i < (int)s.size(); i++) {
            if (s[i] < '0' || s[i] > '9') {
                return false;
            }
        }

        return true;
    }

    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] < '0' || s[i] > '9') {
            return false;
        }
    }

    return true;
}

int main() {
    stack<vector<string>> history_stack;
    map<string, stack<long long>> scope_map;

    history_stack.push(vector<string>());

    string line;
    while (getline(cin, line)) {
        if (line == "{") {
            history_stack.push(vector<string>());
            continue;
        }

        if (line == "}") {
            for (const string& elem : history_stack.top()) {
                scope_map[elem].pop();
            }

            history_stack.pop();
            continue;
        }

        size_t equals_pos = line.find('=');
```

```

string left_arg = line.substr(0, equals_pos);
string right_arg = line.substr(equals_pos + 1);

if (is_number(right_arg)) {
    long long value = stoll(right_arg);
    scope_map[left_arg].push(value);
    history_stack.top().push_back(left_arg);
} else {
    auto& st = scope_map[right_arg];
    long long value;
    if (st.empty()) {
        value = 0;
    } else {
        value = st.top();
    }

    scope_map[left_arg].push(value);
    history_stack.top().push_back(left_arg);
    cout << value << endl;
}
}
}

```

О решении:

Идея:

В качестве истории хранения значений переменных будем использовать словарь стеков, где названия переменных — это ключи, а их значения — стеки. У нас полностью сохраняется история изменения значений всех переменных. Изначально мы на нулевом уровне (“глобальная” видимость), при встрече фигурной скобки переходим на новый уровень видимости: будем в отдельный стек складывать массив, который хранит в себе названия переменных, значения которых были изменены (вписываем их туда, когда их значения меняются). Тогда при встрече обратной фигурной скобки достаём этот массив, а потом будем доставать со стеков словаря значения элементов, которые встретились в массиве. Когда массив опустеет, достанем его с вершины стека. По окончании программы стек массивов будет содержать один пустой массив (для “глобальной” видимости). Если переменная несколько раз перезаписывается, то она несколько раз пушится как в стек в словаре, так и в массив изменений в стеке (что корректно).

Сложность по времени:

$O(n \log k)$ — так как доступ к сбалансированному дереву за $\log k$ (k — количество переменных), который в худшем случае делаем n (количество строк) раз.

Сложность по памяти:

$O(n)$ — так как храним значения максимум n переменных из n строк.

Д. Профессор Хаос

Условие:

В секретной лаборатории профессора Хаоса проходит эксперимент по выращиванию особо опасных бактерий. В начале первого дня эксперимента у Хаоса имеется a особо опасных бактерий.

Каждый день эксперимента устроен следующим образом. Рано утром профессор достает из контейнера все свои бактерии и помещает их в инкубатор, где бактерии начинают делиться. Вместо каждой бактерии образуется b новых бактерий.

После извлечения бактерий из инкубатора c из них используются для проведения различных опытов и затем уничтожаются. Если после извлечения из инкубатора имеется менее c бактерий, для проведения опытов используются все имеющиеся бактерии, и эксперимент заканчивается.

Оставшиеся бактерии в конце дня необходимо поместить в контейнер и продолжить использовать в эксперименте. Однако в контейнер можно поместить не более d бактерий, поэтому если число оставшихся бактерий больше d , то в контейнер помещаются d бактерий, а остальные уничтожаются.

Теперь профессор Хаос хочет выяснить, сколько особо опасных бактерий будет у него в контейнере после k -го дня эксперимента. Помогите ему найти ответ на этот вопрос.

Формат ввода

В единственной строке входного файла содержится пять целых чисел a, b, c, d и k ($1 \leq a, b \leq 1000, 0 \leq c \leq 1000, 1 \leq d \leq 1000, a \leq d, 1 \leq k \leq 10^{18}$).

Формат вывода

Выведите одно число — количество бактерий у Хаоса к концу k -го дня. Если эксперимент завершится в k -й день или ранее, выведите число 0.

Пример 1

Ввод 

1 3 1 5 2

Вывод 

5

Пример 2

Ввод 

1 2 0 4 3

Вывод 

4

Пример 3

Ввод 

1 2 3 5 2

Вывод 

0

Решение:

```
#include <iostream>
#include <vector>

using namespace std;

int day_iteration(int bac_count, int bac_multiplier, int bac_exp, int capacity) {
    long long curr_bac_count = bac_count;

    curr_bac_count *= bac_multiplier;

    if (curr_bac_count < bac_exp) {
        return 0;
    }

    curr_bac_count -= bac_exp;

    if (curr_bac_count > capacity) {
        curr_bac_count = capacity;
    }

    return (int)curr_bac_count;
}

int main() {
    int bac_count, bac_multiplier, bac_exp, capacity;
    long long days;
    cin >> bac_count >> bac_multiplier >> bac_exp >> capacity >> days;

    vector<vector<int>>> jump_table(60, vector<int>(capacity + 1));

    for (int i = 0; i <= capacity; i++) {
        jump_table[0][i] = day_iteration(i, bac_multiplier, bac_exp, capacity);
    }

    for (int p = 1; p < 60; p++) {
        for (int i = 0; i <= capacity; i++) {
            jump_table[p][i] = jump_table[p - 1][jump_table[p - 1][i]];
        }
    }

    int curr_bac_count = bac_count;
    for (int p = 0; p < 60; p++) {
        if ((days & (1LL << p)) != 0) {
            curr_bac_count = jump_table[p][curr_bac_count];
        }
    }

    cout << curr_bac_count << endl;
}
```

О решении:

Идея:

Из-за большого числа дней (10^{18}) мы вылетаем по тайм лимиту, если будем просто итеративно вычислять значение функции. Воспользуемся бинарным подъемом (binary lifting). Пусть функция вычисления количества бактерий к концу дня – $f(i)$. Тогда нетрудно понять, что вычисление количества бактерий на следующий день – это эта же функция от нового значения, то есть $f(f(i))$. Таким образом, чтобы узнать количество бактерий на день k нам надо рассчитать $f(f(f(f(\dots))))$, или же $f^k(i)$. Создадим таблицу переходов (количество строк – 60, так как $2^{60} > 10^{18}$, а длина – вместимость контейнера).

Чтобы получить значение для дня 2^p , получим значение для дня $2^{(p-1)}$, а потом от полученного значения получим значение также для дня $2^{(p-1)}$ (так как $2^{(p-1)} + 2^{(p-1)} = 2^p$). То есть $f^{(2^p)}(i) = f^{(2^{(p-1)})}(f^{(2^{(p-1)})}(i))$. Таким образом композицию функций первого порядка для какого-то числа дней k можем представить в виде композиции функций порядков степеней двойки.

Функция $f(i)$ зависит только от текущего количества бактерий и не зависит от дня, а также количество бактерий никогда не превысит вместимость контейнера, что позволяет просчитать все значения для таблицы переходов. Каждая строка p таблицы хранит значения функции $f^{(2^p)}$ для всех возможных количеств бактерий.

Любое число дней k можно представить в виде суммы степеней двойки, поэтому задача получения количества бактерий на день k преобразуется в следующее условие:

Если бит установлен в единицу в довичном представлении k , то совершаем переход на этот уровень p , и обновляем текущее количество бактерий (изначально – начальное количество). Таким образом получаем композицию функций $f^{(2^p)}$, где p – индексы единиц в битовом представлении k (от младших к старшим). Если на каком-то этапе количество бактерий стало 0, то до конца оно остается таким, так как $f(0)=0$.

Сложность по времени:

$O(d \log k)$ – так как заполняем таблицу длиной d и высотой $\log k$ (но у нас примерно $\log(10^{18})=60$, поэтому фактически время за $O(d)$)

Сложность по памяти:

$O(d \log k)$ – размер таблицы d на $\log(10^{18})$ максимум, фактически $O(d)$